# Chapter 5: Deadlocks

- A situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process .

- In operating systems, managing resources like memory, files, and processors is very important. Sometimes, processes (or programs) get stuck waiting for each other to release resources, leading to a situation called a deadlock.
- To handle deadlocks, operating systems use special methods called deadlock detection algorithms.
- These algorithms help identify when a deadlock has happened so that the system can fix the problem.
- This is different from methods that try to prevent deadlocks from happening at all, which can be more restrictive.

# Overview

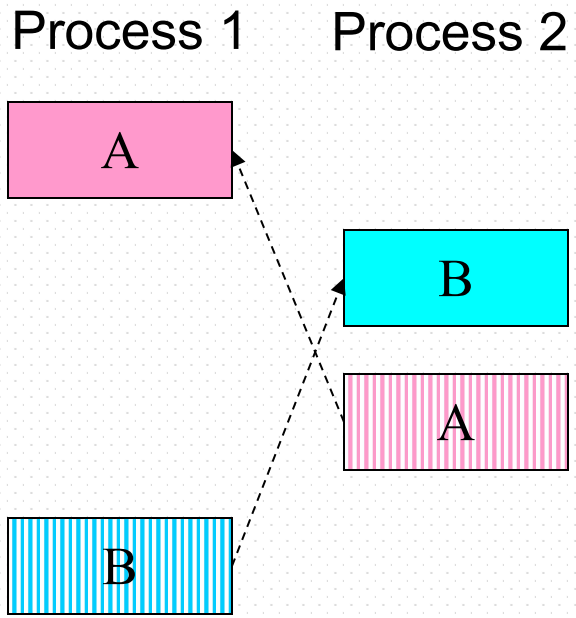
- Resources
- Why do deadlocks occur?
- Dealing with deadlocks
  - Ignoring them: ostrich algorithm
  - Detecting & recovering from deadlock
  - Avoiding deadlock
  - Preventing deadlock

# Resources

- **Resource:** something a process uses
  - Usually limited (at least somewhat)
- Examples of computer resources
  - Printers
  - Semaphores / locks
  - Tables (in a database)
- Processes need access to resources in reasonable order
- **Two types of resources:**
  - Pre-emptable resources: can be taken away from a process with no effects
  - Non pre-emptable resources: will cause the process to fail if taken away

# When do deadlocks happen?

- **Suppose**
  - Process 1 holds resource A and requests resource B
  - Process 2 holds B and requests A
  - Both can be blocked, with neither able to proceed
- **Deadlocks occur when …**
  - Processes are granted exclusive access to devices or software constructs (resources)
  - Each deadlocked process needs a resource held by another deadlocked process




**DEADLOCK!**

# Using resources

- **Sequence of events required to use a resource**
  - Request the resource
  - Use the resource
  - Release the resource
- **Can't use the resource if request is denied**
  - **Requesting process has options**
    - Block and wait for resource
    - Continue (if possible) without it: may be able to use an alternate resource
    - Process fails with error code
  - **Some of these may be able to prevent deadlock…**

# What is a deadlock?

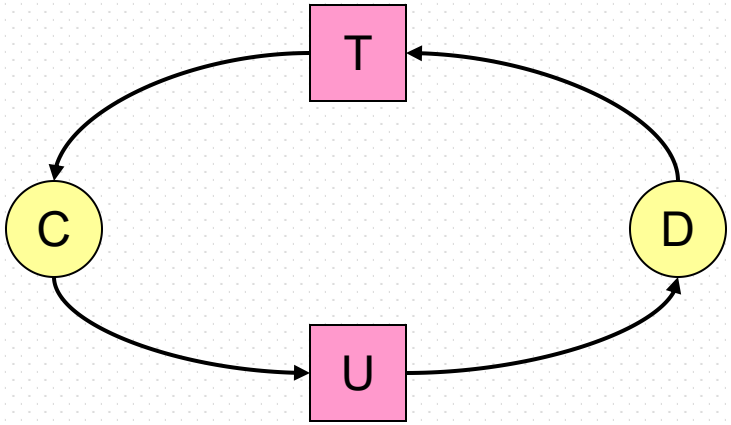
- **Formal definition:**
  **"A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause."**
- **Usually, the event is release of a currently held resource**
- **In deadlock, none of the processes can**
  - Run
  - Release resources
  - Be awaken

# Four conditions for deadlock

- **Mutual exclusion**
  - Each resource is assigned to at most one process
- **Hold and wait**
  - A process holding resources can request more resources
- **No preemption**
  - Previously granted resources cannot be forcibly taken away
- **Circular wait**
  - There must be a circular chain of 2 or more processes where each is waiting for a resource held by the next member of the chain

# Resource allocation graphs




- **Resource allocation modeled by directed graphs**
- **Example 1:**
  - Resource R assigned to process A
- **Example 2:**
  - Process B is requesting / waiting for resource S
- **Example 3:**
  - Process C holds T, waiting for U
  - Process D holds U, waiting for T
  - C and D are in deadlock!

# Dealing with deadlock

**How can the OS deal with deadlock?**

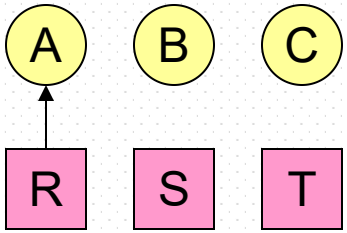
- Ignore the problem altogether!
  - Hopefully, it'll never happen…
- Detect deadlock & recover from it
- Dynamically avoid deadlock
  - Careful resource allocation
- Prevent deadlock
  - Remove at least one of the four necessary conditions
- We'll explore these tradeoffs

# Getting into deadlock

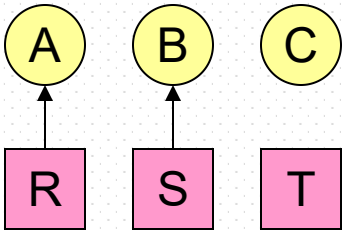
A
Acquire R
Acquire S
Release R
Release S

B
Acquire S
Acquire T
Release S
Release T

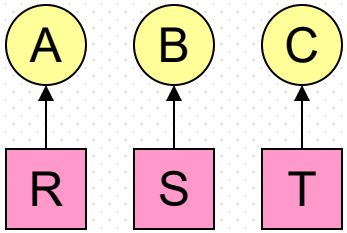
C
Acquire T
Acquire R
Release T
Release R



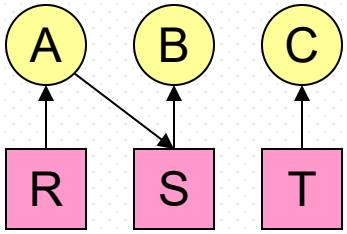

Acquire R



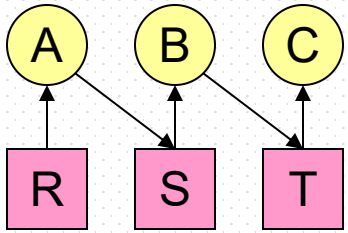

Acquire S



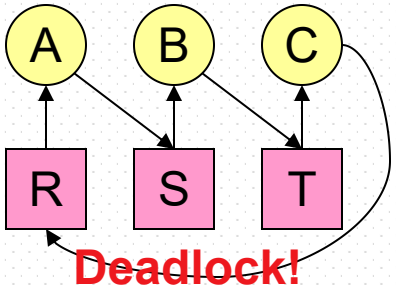

Acquire T




Acquire S




Acquire T




Acquire R

# Not getting into deadlock…

- **Many situations *may* result in deadlock (but don't have to)**
  - In previous example, A could release R before C requests R, resulting in no deadlock
  - Can we always get out of it this way?
- **Find ways to:**
  - Detect deadlock and reverse it
  - Stop it from happening in the first place

# Deadlock Detection Algorithm

- A deadlock detection algorithm is a technique used by an operating system to identify deadlocks in the system. This algorithm checks the status of processes and resources to determine whether any deadlock has occurred and takes appropriate actions to recover from the deadlock.
- Deadlock detection algorithms work by regularly checking the current state of resource usage in the system. They often use a visual tool called a resource allocation graph. This graph shows which processes are using which resources and which resources each process is waiting for. By looking for cycles in this graph, the algorithm can spot deadlocks.

# The algorithm employs several times varying data structures:

- **Available**: A vector of length m indicates the number of available resources of each type.
- **Allocation:** An n*m matrix defines the number of resources of each type currently allocated to a process. The column represents resource and rows represent a process.
- **Request:** An n*m matrix indicates the current request of each process. If request[i][j] equals k then process P i is requesting k more instances of resource type R j .

# Other Algorithm for Deadlock Detection

- **Wait-For Graph:** A graphical representation of the system's processes and resources. A directed edge is created from a process to a resource if the process is waiting for that resource. A cycle in the graph indicates a deadlock.
- **Banker's Algorithm**: A resource allocation algorithm that ensures that the system is always in a safe state, where deadlocks cannot occur.
- **Resource Allocation Graph** : A graphical representation of processes and resources, where a directed edge from a process to a resource means that the process is currently holding that resource. Deadlocks can be detected by looking for cycles in the graph.
- **Detection by System Modeling**: A mathematical model of the system is created, and deadlocks can be detected by finding a state in the model where no process can continue to make progress.
- **Timestamping :** Each process is assigned a timestamp, and the system checks to see if any process is waiting for a resource that is held by a process with a lower timestamp.

# Deadlock detection algorithm

- **General idea: try to find cycles in the resource allocation graph**
- **Algorithm: depth-first search at each node**
  - Mark arcs as they're traversed
  - Build list of visited nodes
  - If node to be added is already on the list, a cycle exists!
- **Cycle == deadlock**

- These algorithms are used in different operating systems and systems with different resource allocation and synchronization requirements. The choice of algorithm depends on the specific requirements of the system and the trade-offs between performance, complexity, and accuracy.

If a system does not employ either a deadlock prevention or deadlock avoidance algorithm then a deadlock situation may occur.

In this case:

- Apply an algorithm to examine the system's state to determine whether deadlock has occurred.
- Apply an algorithm to recover from the deadlock.

# The Ostrich Algorithm

- **Pretend there's no problem. Reasonable if**
  - Deadlocks occur very rarely
  - Cost of prevention is high than let it happen or assume that no problem will ocurs.
- **UNIX and Windows take this approach**
  - Resources (memory, CPU, disk space) are plentiful
  - Deadlocks over such resources rarely occur
  - Deadlocks typically handled by rebooting
- **Trade off between convenience and correctness**

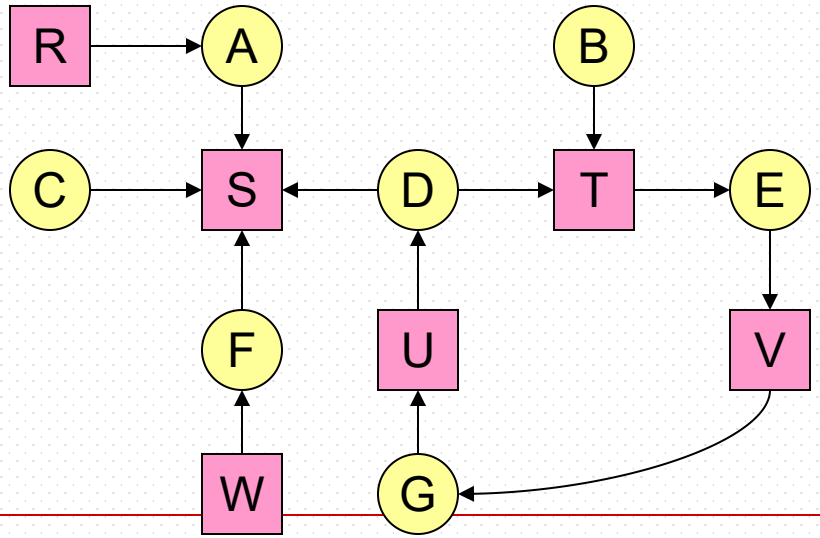


- Yet since the algorithm directly deals with extreme cases it is not a

# Detecting deadlocks using graphs

- **Process holdings and requests in the table and in the graph (they're equivalent)**
- **Graph contains a cycle => deadlock!**
  - Easy to pick out by looking at it (in this case)
  - Need to mechanically detect deadlock
- **Not all processes are deadlocked (A, C, F not in deadlock)**

| Process | Holds | Wants |
|---|---|---|
| A | R | S |
| B | | T |
| C | | S |
| D | U | S,T |
| E | T | V |
| F | W | S |
| G | V | U |



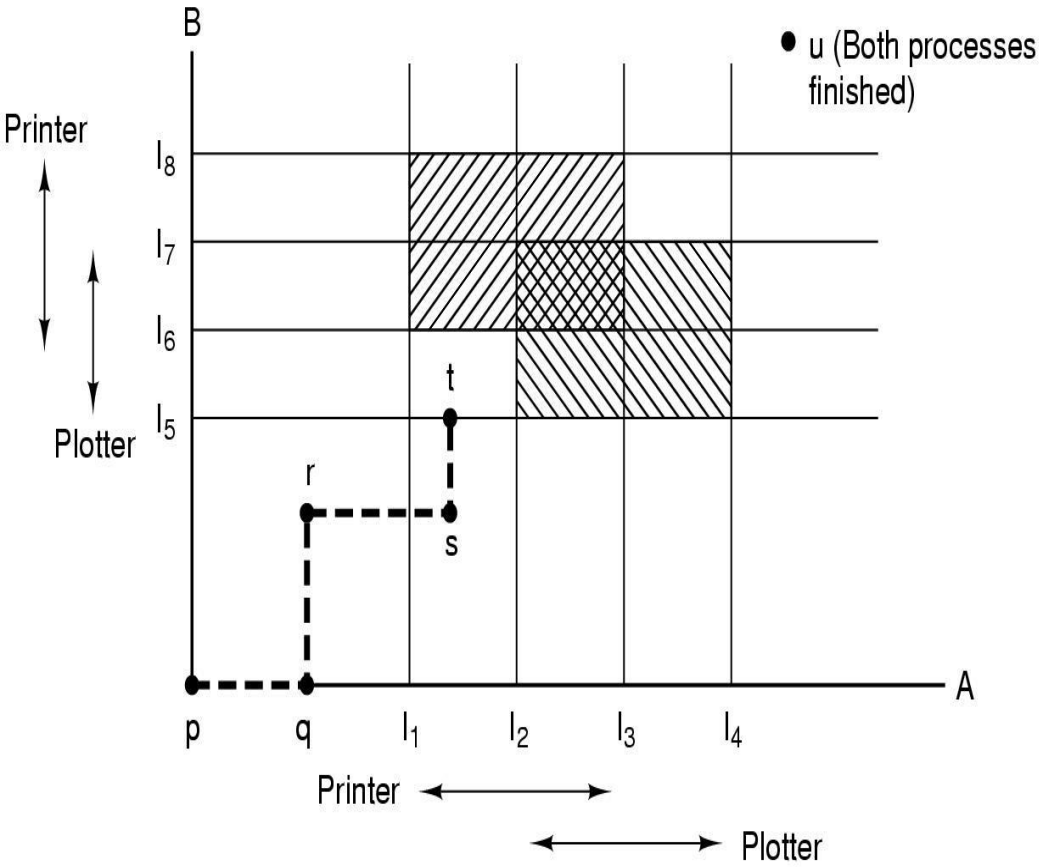
# Resources with multiple instances

- **Previous algorithm only works if there's one instance of each resource**
- **If there are multiple instances of each resource, we need a different method**
  - Track current usage and requests for each process
  - To detect deadlock, try to find a scenario where all processes can finish
  - If no such scenario exists, we have deadlock

# Recovering from deadlock

- **Recovery through preemption**
  - Take a resource from some other process
  - Depends on nature of the resource and the process
- **Recovery through rollback**
  - Checkpoint a process periodically
  - Use this saved state to restart the process if it is found deadlocked
  - May present a problem if the process affects lots of “external” things
- **Recovery through killing processes**
  - Crudest but simplest way to break a deadlock: kill one of the processes in the deadlock cycle
  - Other processes can get its resources
  - Preferably, choose a process that can be rerun from the beginning

# Resource trajectories




Two process resource trajectories

# Preventing deadlock

- **Deadlock can be completely prevented!**
- **Ensure that at least one of the conditions for deadlock never occurs**
  - **Mutual exclusion:** one or more than one resource are no shareble (only one process can use at time)
  - **Hold & wait**: a process is holding at least one resource and waiting for another.
  - **No preemption**: a resource can not be taken from a process unless the process release the resource.
  - **Circular wait** : a set of process are waiting for each other in circular form
- **Not always possible…**

# Eliminating mutual exclusion

- **Some devices (such as printer) can be spool**
  - Only the printer daemon uses printer resource
  - This eliminates deadlock for printer
- **Not all devices can be spooled**
- **Principle:**
  - Avoid assigning resource when not absolutely necessary
  - As few processes as possible actually claim the resource

# Attacking “hold and wait”

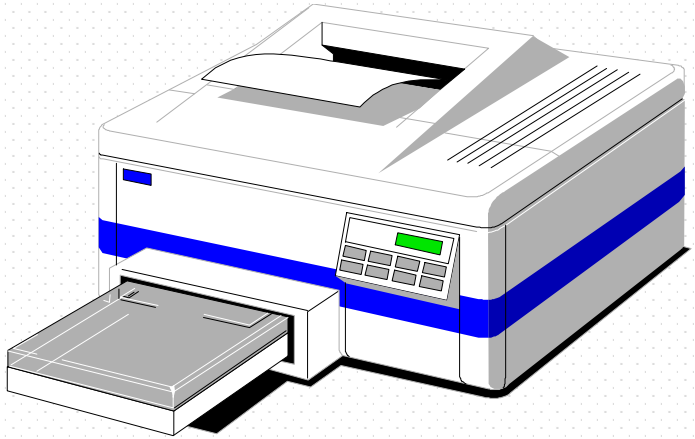
- **Require processes to request resources before starting**
  - A process never has to wait for what it needs
- **This can present problems**
  - **A process may not know required resources at start of run**
  - **This also ties up resources other processes could be using**
    - Processes will tend to be conservative and request resources they might need
- **Variation: a process must give up all resources before making a new request**
  - Process is then granted all prior resources as well as the new ones
  - Problem: what if someone grabs the resources in the meantime—how can the process save its state?

# Example: two-phase locking

- **Phase One**
  - Process tries to lock all data it needs, one at a time
  - If needed data found locked, start over
  - (no real work done in phase one)
- **Phase Two**
  - Perform updates
  - Release locks
- **Note similarity to requesting all resources at once**
- **This is often used in databases**
- **It avoids deadlock by eliminating the "hold-and-wait" deadlock condition**

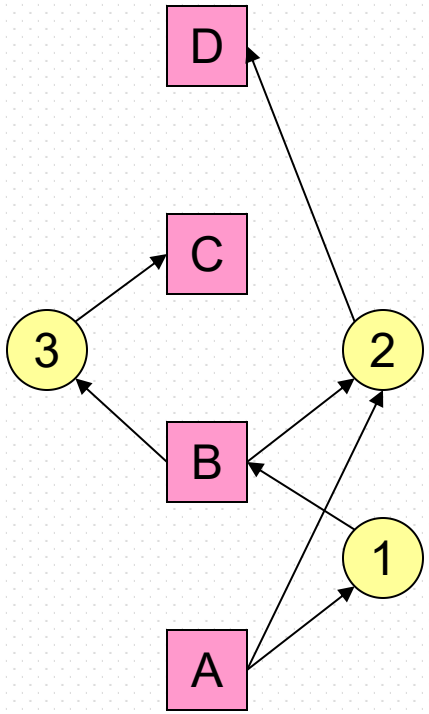
# Attacking “no preemption”

- **This is not usually a viable option**
- **Consider a process given the printer**
  - Halfway through its job, take away the printer
  - Confusion ensues!
- **May work for some resources**
  - Forcibly take away memory pages, suspending the process
  - Process may be able to resume with no ill effects



# Attacking "circular wait"

- **Assign an order to resources**
- **Always acquire resources in numerical order**
  - Need not acquire them all at once!
- **Circular wait is prevented**
  - A process holding resource *n* can't wait for resource *m* if *m* < *n*
  - **No way to complete a cycle**
    - Place processes above the highest resource they hold and below any they're requesting
    - All arrows point up!



# Deadlock prevention: summary

- **Mutual exclusion**
  - Spool everything
- **Hold and wait**
  - Request all resources initially
- **No preemption**
  - Take resources away
- **Circular wait**
  - Order resources numerically

# “Non-resource” deadlocks

- **Possible for two processes to deadlock**
  - Each is waiting for the other to do some task
- **Can happen with semaphores**
  - Each process required to do a down() on two semaphores (mutex and another)
  - If done in wrong order, deadlock results
- **Semaphores could be thought of as resources…**

# Starvation

- **Algorithm to allocate a resource**
  - Give the resource to the shortest job first
- **Works great for multiple short jobs in a system**
- **May cause long jobs to be postponed indefinitely**
  - Even though not blocked
- **Solution**
  - First-come, first-serve policy
- **Starvation can lead to deadlock**
  - Process starved for resources can be holding resources
  - If those resources aren't used and released in a timely fashion, shortage could lead to deadlock

# Advantages of Deadlock Detection Algorithms

- **Improved System Stability**: Deadlocks are a major concern in operating systems , and detecting and resolving deadlocks can help to improve the stability of the system.
- **Better Resource Utilization:** By detecting deadlocks and freeing resources, the operating system can ensure that resources are efficiently utilized and that the system remains responsive to user requests.
- **Easy Implementation:** Some deadlock detection algorithms, such as the Wait-For Graph , are relatively simple to implement and can be used in a wide range of operating systems and systems with different resource allocation and synchronization requirements.

# Disadvantages of Deadlock Detection Algorithms

- **Performance Overhead**: Deadlock detection algorithms can introduce a significant overhead in terms of performance, as the system must regularly check for deadlocks and take appropriate action.
- **Complexity**: Some deadlock detection algorithms, such as the Resource Allocation Graph or Timestamping, are more complex to implement and require a deeper understanding of the system and its behavior.
- **False Positives and Negatives**: Deadlock detection algorithms are not perfect and may produce false positives or negatives, indicating the presence of deadlocks when they do not exist or failing to detect deadlocks that do exist.

Overall, the choice of deadlock detection algorithm depends on the specific requirements of the system, the trade-offs between performance, complexity, and accuracy, and the risk tolerance of the system. The operating system must balance these factors to ensure that deadlocks are detected and resolved effectively and efficiently.

# Conclusion

- Deadlock detection algorithms are important for identifying and resolving situations where processes get stuck waiting for resources held by each other. By regularly checking resource usage and identifying cycles of dependencies, these algorithms help maintain system stability and efficiency. This ensures that processes can run smoothly without being interrupted by deadlocks.

- **END**